

Control-Flow Graphs and Statement, Branch, and Path Coverage

SE 413/513 — Software Engineering

Course Reading Material

Unit: White-Box Testing and Structural Coverage Criteria

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.

For permissions or questions regarding the use of this material, contact hasaka@gvsu.edu.

Please cite [1] when using, adapting, or reproducing this material. The corresponding BibTeX entry is provided below:

```
@misc{hasan2026stesting,
author = {Hasan, Kamrul},
year = {2026},
title = {Software Testing},
institution = {Grand Valley State University},
url = {https://github.com/mdkamrulhasan/software-testing}
}
```

Contents

1	Learning Objectives	3
2	From Black-Box to White-Box Testing	3
3	Control-Flow Graphs	3
3.1	What Is a Control-Flow Graph?	3
3.2	Building a CFG: A Worked Example	3
3.3	CFGs and Loops	4
4	Statement Coverage	5
4.1	Definition	5
4.2	Worked Example	5
4.3	The Limitation of Statement Coverage	5
5	Branch (Decision) Coverage	7
5.1	Definition	7
5.2	What Branch Coverage Catches That Statement Coverage Misses	7
6	Path Coverage	7
6.1	Definition	7
6.2	Why Path Coverage Explodes With Loops	8
6.3	Cyclomatic Complexity and Basis Path Testing	8

7	The Subsumption Hierarchy	8
8	Measuring Coverage in Practice	8
9	What Coverage Cannot Tell You	9
10	Common Pitfalls	10
11	A Practical Mental Model	10
12	Summary	11
13	Practice Exercises	11
14	Discussion Questions	12
15	Key Takeaways for SE 413/513	12
16	References	13

1 Learning Objectives

After completing this reading, you should be able to:

- Explain the difference between black-box and white-box test design, and where coverage fits.
- Construct a control-flow graph (CFG) from a function's source code.
- Define statement, branch, and path coverage, and compute each for a small function.
- Explain why 100% statement coverage does not imply 100% branch coverage.
- Explain why path coverage is often infeasible for code containing loops.
- Compute cyclomatic complexity and relate it to the minimum number of tests needed for branch coverage.
- Measure coverage for a real Python test suite using `coverage.py` / `pytest-cov`.
- Explain what coverage does and does not tell you about correctness.

2 From Black-Box to White-Box Testing

The *Equivalence Partitioning* reading introduced **black-box** test design: choosing inputs based only on a function's specification, without looking at its implementation. This reading turns to **white-box** (also called structural or code-based) testing: deriving test-adequacy criteria by looking directly at a program's internal structure.[2]

Key Idea

Black-box and white-box testing answer two different questions. Black-box design (equivalence partitioning, boundary-value analysis) asks: *"have I chosen a good, representative set of inputs based on what this code is supposed to do?"* White-box **coverage** asks a narrower, complementary question: *"did my tests actually execute the code I wrote?"* A test suite can do very well on one and poorly on the other — and, as this reading will show, doing well on both is still not the same as being correct.

3 Control-Flow Graphs

3.1 What Is a Control-Flow Graph?

A **control-flow graph (CFG)** is a directed graph representation of a program's execution paths. Each **node** represents a basic block — a straight-line sequence of statements with no branches in or out except at its beginning and end. Each **edge** represents a possible transfer of control from one block to another: falling through to the next statement, taking a branch, or looping back.[2]

3.2 Building a CFG: A Worked Example

Consider a function that classifies a financial transaction:

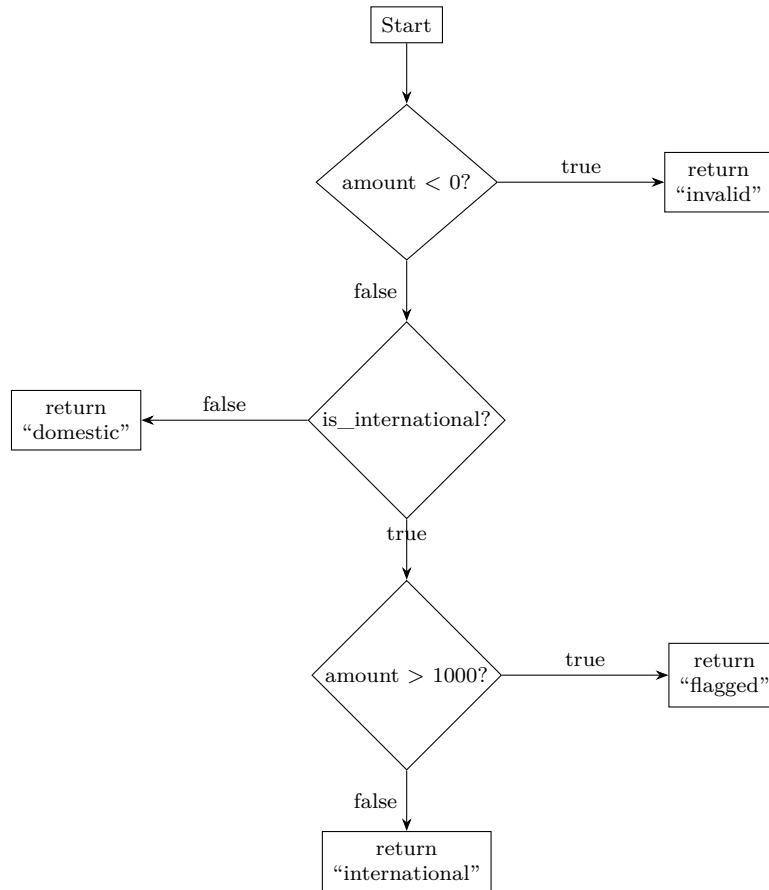
```
def classify_transaction(amount, is_international):
    if amount < 0:
        return "invalid"
    if is_international:
        if amount > 1000:
            return "flagged"
        else:
            return "international"
```

```

else:
    return "domestic"

```

Every `if` is a **decision node**: control can leave it along two different edges, depending on whether the condition is true or false. The resulting control-flow graph is:



Every diamond is a decision node with two outgoing edges; every rectangle is a terminal basic block (here, each ends in a `return`). Reading the graph top to bottom, each root-to-leaf path corresponds to exactly one possible execution of the function for some input.

3.3 CFGs and Loops

Loops introduce a **back edge**: an edge from later in the graph pointing back to an earlier node.

```

def contains_negative(numbers):
    for n in numbers:
        if n < 0:
            return True
    return False

```

Here, the loop condition (“are there more elements?”) is itself a decision node with an edge back to the top of the loop body and an edge out to `return False`; the inner `if` is a second decision node nested inside the loop. Unlike the tree-shaped graph in Section 3.2, this graph contains a cycle — which turns out to matter a great deal once we start counting how many distinct paths exist through it (Section 6).

4 Statement Coverage

4.1 Definition

Statement coverage measures the percentage of executable statements in the program that were executed by at least one test in the suite:[2]

$$\text{Statement coverage} = \frac{\text{statements executed by at least one test}}{\text{total executable statements}} \times 100\%$$

4.2 Worked Example

`classify_transaction()` has seven executable statements: three `if` checks and four `return` statements. For this function, the single test case `classify_transaction(-5, False)` executes the `amount < 0` check and the `return "invalid"` statement — but none of the other five statements. A second test, `classify_transaction(50, True)`, additionally executes the `is_international` check, the `amount > 1000` check, and `return "international"`. Even with both tests, `return "flagged"` and `return "domestic"` are never executed — 5 of the function's 7 statements have run, so statement coverage is $5/7 \approx 71\%$. A third test, `classify_transaction(50, False)`, and a fourth, `classify_transaction(5000, True)`, bring every statement to 100%.

4.3 The Limitation of Statement Coverage

Statement coverage has a well-known blind spot: it says nothing about whether both outcomes of a decision were exercised. Consider:

```
def apply_membership_discount(price, is_member):  
    if is_member:  
        price = price * 0.9  
    return price
```

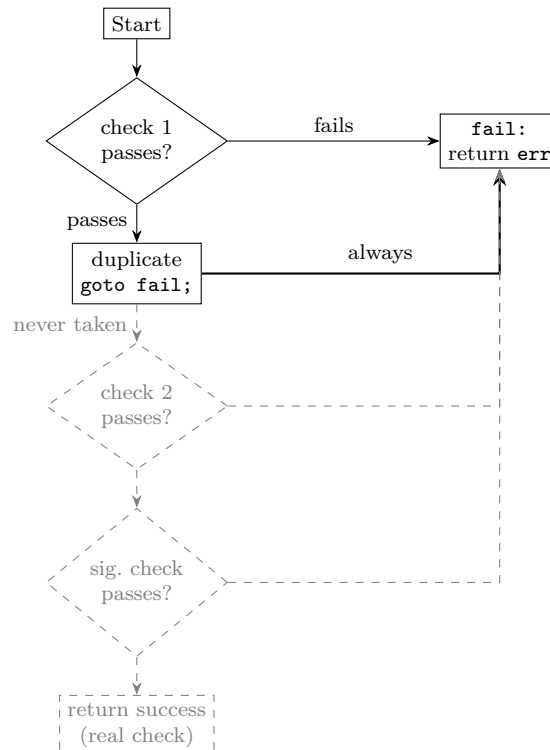
A single test, `apply_membership_discount(100, True)`, executes *every statement* in this function — 100% statement coverage — while never once executing the case where `is_member` is `False`. If a future edit accidentally breaks the non-member path (say, applying the discount unconditionally, or raising an exception when `is_member` is falsy), a statement-coverage-complete suite that never tried `is_member=False` would not notice.

Real-World Case Study

The “goto fail” Bug (Apple, CVE-2014-1266, February 2014).[6] Apple’s SecureTransport library (used by iOS and OS X, including Safari) contained a TLS certificate-verification function with a chain of checks, each of the form `if (error) goto fail;`. A duplicated line broke the pattern:

```
if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
    goto fail;
    goto fail; // duplicate line, NOT inside the if (no braces)
if ((err = SSLHashSHA1.update(&hashCtx, &hashOut)) != 0)
    goto fail; // <-- this check, and everything after it,
                //      can now never execute
...
fail:
    return err;
```

Because the duplicated `goto fail;` has no enclosing braces, it runs *unconditionally* right after the first check passes — jumping straight to the `fail:` label and returning `err`, which at that point still holds 0 (success). Every remaining check, including the one that actually verified the cryptographic signature, became **unreachable code**: no input, however constructed, could ever cause it to execute. The practical impact was severe — an attacker could present an invalid signature and SecureTransport would still report the connection as successfully verified, enabling man-in-the-middle attacks against TLS traffic. The diagram below shows the resulting control-flow graph; the shaded region is unreachable from `Start` under any input.



Tip

This exact bug shape — a statement whose indentation *suggests* it belongs to the `if` block above it, but actually does not, because C treats indentation as cosmetic — cannot occur in Python, where indentation *is* the block structure. That does not make Python immune to the underlying lesson: an early `return` that accidentally short-circuits later validation is easy to write in any language, and unreachable code is exactly the kind of defect that a statement-coverage report makes visible immediately (any modern coverage tool would have shown the final signature check at 0% executed, under *any* test).

5 Branch (Decision) Coverage

5.1 Definition

Branch coverage (also called decision coverage) measures the percentage of decision outcomes — both the true and the false direction of every decision — that were exercised by at least one test:[2]

$$\text{Branch coverage} = \frac{\text{branch outcomes taken by at least one test}}{\text{total branch outcomes}} \times 100\%$$

5.2 What Branch Coverage Catches That Statement Coverage Misses

Return to `apply_membership_discount()`. It has one decision (`if is_member`), with two outcomes: true and false. The single test `apply_membership_discount(100, True)` exercises only the true outcome — 1 of 2 branches, 50% branch coverage, even though it already reached 100% statement coverage. Reaching 100% branch coverage requires a second test, `apply_membership_discount(100, False)`, that specifically drives the condition false.

Key Idea

Branch coverage **subsumes** statement coverage: any test suite that achieves 100% branch coverage is guaranteed to achieve 100% statement coverage too (every statement lies on some branch outcome), but not the reverse. This is why branch coverage is generally considered a stronger, more meaningful criterion, and why most industrial coverage tools report it by default alongside (or instead of) plain statement coverage.

Tip

A useful habit: for every `if`, `elif`, `while`, and boolean short-circuit (`and/or`) in the code, ask “do I have a test that makes this true, *and* a separate test that makes it false?” A quick way to sanity-check a suite: an `if` with no matching `else` anywhere in the codebase is a common place branch coverage catches a gap that statement coverage would not.

6 Path Coverage

6.1 Definition

Path coverage measures the percentage of distinct, complete paths through the control-flow graph — from entry to exit — that were exercised by at least one test.[2] It is the strongest of the three criteria: covering every path guarantees covering every branch, which guarantees covering every statement.

For `classify_transaction()` (Section 3.2), there are exactly four paths through the graph, corresponding to the four test cases used to reach 100% statement coverage in Section 4.2. Achieving 100% path coverage there required only four tests, because the graph has no loops.

6.2 Why Path Coverage Explodes With Loops

Now reconsider `contains_negative()` from Section 3.3. The number of distinct paths depends on how many times the loop body executes: zero iterations (empty list), one iteration that returns `True` immediately, one iteration that does not, two iterations, and so on. A list of length n contributes up to 2^n distinct paths, depending on where (if anywhere) a negative number appears. For an unbounded input list, the number of possible paths is unbounded — **exhaustive path coverage is generally infeasible for any code containing a loop whose iteration count depends on input.**

6.3 Cyclomatic Complexity and Basis Path Testing

Since testing every path is usually impossible, Thomas McCabe proposed a practical compromise: **cyclomatic complexity**, a metric that counts the number of *linearly independent* paths through a CFG, computed as[3]

$$V(G) = E - N + 2$$

where E is the number of edges and N is the number of nodes in the CFG (for a single-entry, single-exit graph). Equivalently, for structured code, $V(G)$ equals the number of decision points plus one.

For `classify_transaction()`: three decisions (`amount < 0`, `is_international`, `amount > 1000`) give $V(G) = 3 + 1 = 4$ — matching the four paths found above exactly, since this graph has no loops. For `contains_negative()`: two decisions (the loop condition and the inner `if`) give $V(G) = 2 + 1 = 3$.

Key Idea

Cyclomatic complexity is useful in two ways. First, it gives a concrete lower bound: $V(G)$ is the *minimum* number of test cases needed to achieve 100% branch coverage. Second, **basis path testing** uses $V(G)$ as a target: rather than testing every one of the (potentially infinite) paths through a loop, choose a set of $V(G)$ independent paths that, together, exercise every edge in the graph at least once — a practical, finite stand-in for full path coverage.

Tip

Cyclomatic complexity is also used, informally, as a maintainability signal, independent of testing: a function with $V(G)$ above roughly 10 is often considered a candidate for refactoring, simply because a human reader has to hold that many independent branches in mind to understand every way the function can behave.

7 The Subsumption Hierarchy

The three criteria in this reading form a strict hierarchy of strength:

$$\text{Path coverage} \implies \text{Branch coverage} \implies \text{Statement coverage}$$

Each arrow means “achieving 100% of the criterion on the left guarantees 100% of the criterion on the right, but not vice versa.” A test suite can reach 100% statement coverage while missing many branches (Section 4.3); it can reach 100% branch coverage while missing many paths through a loop (Section 6.2); path coverage is the strongest guarantee, but frequently the most expensive or outright infeasible to achieve exhaustively.

8 Measuring Coverage in Practice

For Python code tested with `pytest`, the `pytest-cov` plugin (built on the `coverage.py` library) measures statement and branch coverage automatically:

```

pip install pytest-cov
pytest --cov=myapp --cov-branch --cov-report=term-missing

```

Running this against a test suite that only exercises the “member” branch of the discount function from Section 5.2 produces output resembling:

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
-----	-----	-----	-----	-----	-----	-----
myapp.py	4	0	2	1	83%	2->4
-----	-----	-----	-----	-----	-----	-----
TOTAL	4	0	2	1	83%	

(Line 1 is the `def`, line 2 is `if is_member:`, line 3 is `price = price * 0.9`, line 4 is `return price`.) The **Missing** column’s 2->4 means: at line 2, the edge that falls straight through to line 4 — skipping line 3, i.e. the `False` branch — was never taken. This is exactly the gap identified by hand in Section 5.2, now surfaced automatically by a tool, in the same style as the coverage reports you will see on course assignments.

Tip

`-cov-branch` matters: by default, `coverage.py` reports statement coverage only, which (as Section 4.3 showed) can look deceptively complete.^[4] Always enable branch coverage when it is available.

9 What Coverage Cannot Tell You

Even 100% path coverage is not a correctness guarantee, for two independent reasons.

First, recall the *Testing Foundations* reading’s Principle 1: tests show the *presence* of defects, not their absence. Coverage only tells you a statement, branch, or path was *executed* — it says nothing about whether the test actually checked that the output was *correct*. A test with no assertion at all (or an assertion that is always true) can achieve 100% coverage of buggy code while catching nothing.

Second, and more subtly: coverage is a purely *structural* criterion, entirely blind to the specific *values* used to reach a given branch. Recall the boundary bug in `determine_shipping_cost()` from the *Equivalence Partitioning* reading, where `elif weight_kg < 5` should have been `<= 5`:

```

elif weight_kg < 5:
    return 10.00
else:
    return 20.00

```

Any test with, say, `weight_kg = 3.0` takes the true branch; any test with `weight_kg = 10.0` takes the false branch. That is enough for **100% branch coverage** on this decision — coverage tools have no concept of “the exact boundary value where this condition flips.” The specific input `weight_kg = 5.0` that actually exposes the fault is invisible to a coverage report; only boundary-value analysis, choosing test inputs based on the specification’s boundaries rather than the code’s branch structure, reliably finds it.

Key Idea

Coverage and value-based techniques like equivalence partitioning and boundary-value analysis are **complementary**, not substitutes. Coverage answers “did I exercise this code at all?”; EP/BVA answers “did I choose the specific inputs most likely to expose a defect?” A mature test suite uses black-box techniques to choose *which* values to test, and white-box coverage to confirm nothing in the implementation was left completely untested.

10 Common Pitfalls

Common Pitfall

Treating a coverage percentage as a quality target in itself. Chasing “100% coverage” can produce tests written purely to execute a line, with weak or missing assertions — satisfying the metric while adding little real defect-detection power. Coverage is a diagnostic for gaps, not a goal to be maximized for its own sake.

Common Pitfall

Reporting statement coverage as if it were branch coverage. As Section 4.3 and Section 5.2 showed, these can differ substantially on the very same code. Always check which criterion a reported percentage actually measures.

Common Pitfall

Assuming high coverage means boundary values were tested. As Section 9 showed, a coverage report cannot distinguish a boundary-value test from an arbitrary interior value that happens to take the same branch. High coverage numbers can create false confidence specifically around off-by-one defects.

11 A Practical Mental Model

When you see a coverage report showing 100% statement coverage, think:

“Every line ran at least once. Did every decision’s *both* outcomes run? Let me check branch coverage specifically.”

When you see an `if` with no corresponding `else` anywhere in the test suite, think:

“This is a branch-coverage gap waiting to be found — what test would drive this condition false?”

When a function contains a loop, think:

“Full path coverage here is likely infeasible. What is the cyclomatic complexity, and what small set of representative paths (zero iterations, one, many) gives good basis-path coverage instead?”

When a coverage report shows 100% and you still suspect a bug, think:

“Coverage confirms this code ran — it says nothing about whether the specific values I used were the ones most likely to expose a defect. Did I also apply boundary-value analysis here?”

12 Summary

Term	Meaning
Control-flow graph (CFG)	A directed graph of basic blocks (nodes) and possible transfers of control (edges)
Basic block	A straight-line sequence of statements with no internal branches
Statement coverage	% of statements executed by at least one test
Branch coverage	% of decision outcomes (true/false) executed by at least one test
Path coverage	% of distinct entry-to-exit paths executed by at least one test
Cyclomatic complexity $V(G)$	Number of linearly independent paths; decision points + 1
Basis path testing	Testing a set of $V(G)$ independent paths as a practical stand-in for full path coverage
Subsumption hierarchy	Path $\Rightarrow$ branch $\Rightarrow$ statement coverage, strongest to weakest

Key Idea

Coverage tells you where your tests didn't look; it never tells you your tests looked hard enough.[2] Use control-flow graphs to reason about a function's structure, use statement and branch coverage to find code your tests never exercise at all, and use cyclomatic complexity and basis path testing to keep looping code testable — but keep pairing all of it with the value-based techniques from the *Equivalence Partitioning* reading, since coverage alone cannot see the difference between a boundary value and an arbitrary one.

13 Practice Exercises

Exercise 1 — Building a CFG

Draw the control-flow graph for the following function, labeling each decision node and each terminal block:

```
def grade_letter(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    else:
        return "F"
```

Exercise 2 — Statement vs. Branch Coverage

For `grade_letter()` above, give a set of test inputs that achieves 100% statement coverage but less than 100% branch coverage. Then extend it to a set that achieves 100% branch coverage.

Exercise 3 — Cyclomatic Complexity

Compute the cyclomatic complexity of `grade_letter()` using the “decision points + 1” rule. How many test cases, at minimum, are required for 100% branch coverage? Does your answer from Exercise 2 match?

Exercise 4 — Path Coverage and Loops

Consider a function that returns the index of the first occurrence of a target value in a list, or -1 if not found, using a `for` loop with an early `return`. Explain, in your own words, why the number of distinct paths through this function grows with the length of the input list, and propose a small, representative set of test cases (using the spirit of basis path testing) rather than attempting exhaustive path coverage.

Exercise 5 — Design Question

A teammate says: “Our test suite has 100% branch coverage, so we can be confident the code is correct.” Using at least one concept from this reading (the subsumption hierarchy, the boundary-value example in Section 9, or the “goto fail” case study), write a short argument explaining what this claim gets right and what it misses.

14 Discussion Questions

1. What is a basic block, and why do control-flow graphs use them as nodes instead of individual statements?
2. Why does 100% statement coverage not guarantee 100% branch coverage? Construct your own small example distinct from this reading’s.
3. Why is branch coverage said to “subsume” statement coverage?
4. Why does a loop make exhaustive path coverage infeasible in general?
5. What does cyclomatic complexity actually count, and why is decision points $+ 1$ a useful shortcut for computing it?
6. How does basis path testing attempt to make path-coverage-like guarantees practical for looping code?
7. In the “goto fail” case study, which coverage criterion most directly would have flagged the defect, and why?
8. Why can a coverage tool not detect that a test used a boundary value rather than an arbitrary interior value?
9. Why might a team deliberately choose not to pursue 100% branch coverage on every module in a codebase?
10. How do control-flow-graph-based coverage criteria and the black-box techniques from the *Equivalence Partitioning* reading complement each other in a real test strategy?

15 Key Takeaways for SE 413/513

1. **A CFG makes a function’s structure explicit.** Nodes are basic blocks; edges are possible transfers of control, including loop back-edges.
2. **Statement, branch, and path coverage form a strict hierarchy.** Each is a stronger, more expensive guarantee than the last.
3. **Statement coverage can hide untested branches.** Always prefer branch coverage when a choice is available.
4. **Loops make exhaustive path coverage generally infeasible.** Cyclomatic complexity and basis path testing offer a practical, finite alternative.
5. **Coverage tools measure execution, not correctness.** A covered line with no meaningful assertion proves little.
6. **Coverage cannot see boundary values.** Structural coverage and value-based techniques (EP, BVA) must be used together.

7. **Unreachable code is a real, findable defect class.** The “goto fail” bug shows how directly a basic coverage report can catch a serious, real-world security flaw.

16 References

- Course material on software testing principles and related testing practices.[1]
- White-box testing, control-flow graphs, statement coverage, and branch coverage.[2]
- Cyclomatic complexity and basis-path testing.[3]
- Python branch-coverage measurement with `coverage.py`. [4]
- `pytest-cov` integration for measuring coverage in pytest-based test suites.[5]
- Documentation of the Apple “goto fail” security defect.[6]

References

- [1] Hasan, K. (2026). *Software Testing*. Grand Valley State University. <https://github.com/mdkamrulhasan/software-testing>
- [2] International Software Testing Qualifications Board (ISTQB). (2024). *Certified Tester Foundation Level (CTFL) Syllabus, v4.0.1*. https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf
- [3] McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320. <https://doi.org/10.1109/TSE.1976.233837>
- [4] Ned Batchelder and contributors. (2026). *coverage.py Documentation: Branch coverage measurement*. <https://coverage.readthedocs.io/en/latest/branch.html>
- [5] pytest-cov contributors. (2026). *pytest-cov Documentation*. <https://pytest-cov.readthedocs.io/>
- [6] Apple Inc. (2014). *About the security content of iOS 7.0.6*. Apple Support. <https://support.apple.com/en-us/HT6147>